

Core Components Assessment

GCSE Computer Science · Python Lab · Archbishop McGrath Catholic High School

Ollie · Class not entered · 10CS · 2026-09-24



22 / 56 marks · 39.3% · 34 not yet achieved · 0 unmarked
Developing · Teacher-reviewed classroom scheme

Developing: 25% (14 marks) · Secure: 60% (34 marks) · Mastery: 80% (45 marks). These are classroom checkpoints, not GCSE grades. Unmarked criteria have not been treated as incorrect.

Assessment breakdown

Task	Achieved	Not yet	Unmarked
A · Knowledge & tracing	4 / 9	5	0
B · Debug lab	2 / 6	4	0
C1 · Merchandise order	8 / 12	4	0
C2 · Priority delivery	6 / 8	2	0
C3 · Booking repair	0 / 9	9	0
C4 · Tournament budget	2 / 12	10	0

A message from your teacher

Ollie, you have shown some strengths in using inputs and basic variables, which gives you a useful foundation to build on. However, this result shows that there are still some important gaps in your understanding.

Your main focus now should be using f-strings confidently to combine text and variables correctly. Make sure you use the workbook tasks to practise this regularly, as this will help strengthen your overall programming skills.

WWW · What went well

Inputs · input()

You collected and stored the values needed by the program.

Evidence: C1.1, C1.2.

Type casting · int() and float()

You selected an appropriate numerical conversion in the recorded work.

Evidence: C1.3, C1.4.

Variables

You used variables to store inputs or calculation results and refer to them later.

Evidence: C1.1, C1.2.

Calculations and tracing

You selected appropriate operators and linked stored values to calculate results.

Evidence: C1.6, C2.2.

EBI · Even better if

F-strings and formatting

Use f before the quotation marks and put variables inside {braces}. Use `:.2f` for monetary output when the task requires two decimal places.

Evidence: C1.10, C2.7.

Naming conventions and comments

Choose meaningful, valid variable names and use one naming convention consistently. Add comments that explain a section's purpose.

Evidence: C1.11, C2.8.

Outputs · print()

Use print() to display each required result. Distinguish literal text in quotes from a variable's stored value.

Evidence: C1.10, C2.7.

Individual mark checklist

Each criterion shows its maximum mark. "Not yet" is a missed mark; "Unmarked" awaits a decision. This is a proposed teacher scheme based on the workbook's questions and requirements, not an official exam-board scheme.

A1 · Question 01 · 0 / 1

Criterion	What was required	Decision
A1.1	Give the correct answer: Boolean.	0 / 1

A2 · Question 02 · 1 / 1

Criterion	What was required	Decision
A2.1	Give the correct answer: firstScore.	1 / 1

A3 · Question 03 · 1 / 1

Criterion	What was required	Decision
A3.1	Give the correct answer: age = int(input("Age: ")).	1 / 1

A4 · Question 04 · 0 / 1

Criterion	What was required	Decision
A4.1	Give the correct answer: 3.	0 / 1

A5 · Question 05 · 0 / 1

Criterion	What was required	Decision
A5.1	Give the correct answer: print(workerName).	0 / 1

A6 · Question 06 · 1 / 1

Criterion	What was required	Decision
A6.1	Give the correct answer: 4.	1 / 1

A7 · Question 07 · 1 / 1

Criterion	What was required	Decision
A7.1	Give the correct answer: Inspector: Sam.	1 / 1

A8 · Question 08 · 0 / 2

Criterion	What was required	Decision
A8.1	Explain that a syntax error breaks Python's language rules and prevents the affected code from running.	0 / 1
A8.2	Explain that a logic error allows a program to run but produces the wrong result.	0 / 1

Teacher note: Not precise enough here!

Try to use examples to support your example.

e.g. A syntax error is when the rules of the coding language are broken. Like missing brackets or quotation marks.

B1 · Debug 01 · Merchandise order · 1 / 3

Criterion	What was required	Decision
B1.1	Convert the quantity input to an integer before using it in arithmetic.	0 / 1
B1.2	Use quantity × unitPrice to calculate the total.	1 / 1
B1.3	Use an f-string to combine the output text with the total variable.	0 / 1

B2 · Debug 02 · Time converter · 1 / 3

Criterion	What was required	Decision
B2.1	Use integer division by 60 to calculate complete hours.	1 / 1
B2.2	Use remainder after division by 60 to calculate the remaining minutes.	0 / 1
B2.3	Produce the correct labelled result: 2 hours and 22 minutes.	0 / 1

C1 · Practical 01 · Gaming merchandise order · 8 / 12

Criterion	What was required	Decision
C1.1	Collect and store the customer's name.	1 / 1
C1.2	Collect and store the product name.	1 / 1
C1.3	Collect quantity and convert it to an integer.	1 / 1
C1.4	Collect unit price and convert it to a float.	1 / 1
C1.5	Calculate subtotal as quantity × unit price.	0 / 1
C1.6	Use a fixed delivery charge of £2.50.	1 / 1
C1.7	Calculate total as subtotal + delivery.	0 / 1
C1.8	Identify the customer, product and quantity in the order summary.	1 / 1
C1.9	Display labelled subtotal, delivery and total values.	1 / 1
C1.10	Format every monetary amount to two decimal places.	0 / 1
C1.11	Add useful section comments.	0 / 1
C1.12	Use meaningful variable names that make the program easy to follow.	1 / 1

Teacher note: Revisit f-strings. These are used in every program. Mastery is key.

C2 · Practical 02 · Priority delivery upgrade · 6 / 8

Criterion	What was required	Decision
C2.1	Retain the existing item, quantity and unit-price inputs with correct types.	1 / 1
C2.2	Retain the correct subtotal calculation.	1 / 1
C2.3	Store a fixed priority-delivery charge of £6.00.	1 / 1
C2.4	Calculate the updated total as subtotal + priority delivery.	1 / 1
C2.5	Display a PRIORITY ORDER heading.	1 / 1
C2.6	Show the item and a clear subtotal, delivery and total breakdown.	1 / 1
C2.7	Display all monetary values to two decimal places.	0 / 1
C2.8	Add a useful comment explaining the new section.	0 / 1

C3 · Practical 03 · Gaming lounge booking repair · 0 / 9

Criterion	What was required	Decision
C3.1	Convert the number of teams to an integer.	0 / 1
C3.2	Convert players per team to an integer.	0 / 1
C3.3	Convert price per player to a float.	0 / 1
C3.4	Calculate seats as teams × players per team.	0 / 1
C3.5	Calculate total price as seats × price per player.	0 / 1
C3.6	Repair the seat output so it displays without a text/number type error.	0 / 1
C3.7	Repair the total-price output so it displays without a type error.	0 / 1
C3.8	Display the total price as money to two decimal places.	0 / 1
C3.9	Add useful comments explaining the repaired program.	0 / 1

C4 · Practical 04 · Sports tournament budget · 2 / 12

Criterion	What was required	Decision
C4.1	Collect and store the tournament name.	1 / 1
C4.2	Collect team count and players per team as appropriate whole numbers.	1 / 1
C4.3	Collect entry price per player as a decimal-capable number.	0 / 1
C4.4	Collect venue hire and officials cost as decimal-capable numbers.	0 / 1
C4.5	Calculate total players as teams × players per team.	0 / 1
C4.6	Calculate entry income as total players × entry price.	0 / 1
C4.7	Calculate total costs as venue hire + officials cost.	0 / 1
C4.8	Calculate the expected balance as income – total costs.	0 / 1
C4.9	Display a complete, clearly labelled budget summary.	0 / 1
C4.10	Format the monetary values to two decimal places.	0 / 1
C4.11	Use purposeful comments to organise the budget program.	0 / 1
C4.12	Use meaningful identifiers throughout the budget program.	0 / 1

Progress rubric

Developing means some requirements achieved; Secure means the key concepts achieved; Mastery means all checklist requirements for that skill achieved. The green tick shows the selected level. Skill levels are separate from the overall checkpoint badge.

Skill	Developing	Secure	Mastery
Inputs · input() Secure	I can collect a value from the user and store it in a variable.	✓ I can use clear prompts to collect and store the main information a program needs.	I can collect and store all the information needed for each program, including names, quantities, prices and costs.
Outputs · print() Secure	I can display text and the value stored in a variable.	✓ I can display the main results with clear labels so the user can understand them.	I can produce complete, clearly labelled summaries that show every requested detail and result, with money formatted where required.
Variables Secure	I can create variables and store values in them.	✓ I can store inputs and calculation results in variables, then reuse those values in calculations and output.	I can assign, update and reuse values accurately throughout my programs, and correctly trace how variable values change.

Skill	Developing	Secure	Mastery
Naming conventions and comments Secure	I can use valid variable names and add a simple comment.	✓ I can choose meaningful names and use comments to identify the main sections of my code.	I can make my programs easy to follow, using meaningful names throughout and useful comments for the sections, changes and repairs requested.
Type casting · int() and float() Developing	✓ I can convert an input into a number using int() or float().	I can use int() for whole-number counts and float() for prices so the values can be used in calculations.	I can apply suitable conversions to every numerical input, handling whole-number quantities and decimal prices and costs correctly.
Calculations and tracing Developing	✓ I can carry out simple arithmetic and follow a short sequence of calculations.	I can choose the correct operators to calculate quantities, subtotals, totals and remainders.	I can build and trace the required calculations accurately from start to finish, including the time conversion and the tournament's income, costs and final balance.
F-strings and formatting Developing	✓ I can recognise an f-string and use one to include a variable in a simple message.	I can combine text and variable values correctly using the f prefix and braces {}.	I can correctly predict and produce f-string output, and display monetary values to two decimal places wherever the task requires it.
Python knowledge and error types Developing	✓ I can recognise basic Python terms, including Boolean values and programming errors.	I can identify Boolean values and distinguish syntax errors from logic errors, although my description of their effects may be incomplete.	I can correctly identify Boolean values and describe how syntax errors break Python's rules and prevent code running, while logic errors produce incorrect results.
Debugging and repair Developing	✓ I can identify and correct a simple mistake in supplied code.	I can repair common mistakes in input types, calculation operators and output statements.	I can fully repair the supplied programs so their inputs, calculations and outputs work together correctly, while retaining the code that already works.

DIRT · Improve your work

[] **Your improvement task:** Improve and complete all sections in this assessment.
Use the support materials in the workbooks and unit 1 digital textbook to support you.

Revisit f-strings in your 1.1 workbook.

What did you change?

You can find my improved work/DIRT task at

DIRT completion does not change the original assessment mark. Keep pupil information private.